LAYERBUDGET: Per-Layer Token-Precision Joint Optimization for KV Cache Compression

Anonymous Authors[1]

Abstract

KV cache compression methods either evict tokens or quantize, but apply their strategy *uniformly* across layers. We present LAYERBUDGET, an online method that *jointly* allocates per-layer token budgets and quantization precision via a greedy marginal-gain allocator. Two design principles drive its effectiveness: (1) *quantize first, evict last*—the allocator exhausts INT4/INT8 headroom before evicting tokens; (2) *protect early layers*—leave-one-out analysis reveals that early layers are the quality bottleneck under eviction, so inverted importance weights assign them higher retention budgets. Evicted positions are filled with the mean of retained tokens, reducing output distortion by 96%. With inverted importance, LAYERBUDGET achieves PPL ratio 1.01/1.28 at 4×/6× on Mistral-7B (zero-fill)—60% better than conventional late-layer weighting—and 1.00/2.52 on Llama-2-7B. In a controlled comparison of 12 baselines, LAYERBUDGET ranks first among methods that perform token-level compression at 2–4×, while matching quantization-only quality. We validate end-to-end across four models as a drop-in HuggingFace cache and demonstrate block-level integration with vLLM's PagedAttention.

1. Introduction

The Key-Value (KV) cache is the primary memory bottleneck in autoregressive LLM inference (Kwon et al., 2023). For a model with L layers, H KV heads of dimension d, and sequence length S, the cache consumes $2LSHd \cdot b$ bytes, where b is the per-element byte-width. At FP16, Llama-2-70B at 8K context requires over 40 GB—exceeding consumer GPU VRAM.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Two families of KV cache compression have emerged:

1. **Token eviction** (H2O (Zhang et al., 2024), SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2024)): drop "unimportant" tokens, reducing effective S.
2. **KV quantization** (KIVI (Liu et al., 2024), KVQuant (Hooper et al., 2024)): reduce bit-width b, lowering bytes per element.

The uniform assumption. Nearly all existing methods apply their strategy *uniformly across layers*—every layer drops the same fraction of tokens or uses the same bit-width. Yet transformer layers exhibit dramatically *heterogeneous* attention patterns. Our profiling experiments (Section 4) show that the Gini coefficient of attention weight distributions ranges from 0.43 to 0.93 across layers in TinyLlama-1.1B—a 2.1× spread. Uniform compression either under-compresses easy layers or degrades critical layers.

Our insight: quantize first, evict last. Token eviction and quantization have *complementary* cost structures. Quantization (INT4) achieves ∼4× compression with <0.4% quality loss (cosine similarity 0.996). Eviction achieves arbitrary compression but degrades quality rapidly—even with mean-fill (filling evicted positions with the mean of retained KV), H2O at 2× degrades 6–16%. The optimal strategy is therefore to *exhaust quantization headroom before evicting tokens*. Moreover, the eviction/quantization sensitivity differs per layer:

- **High-sparsity layers** (high Gini coefficient): attention is concentrated on a few tokens. These layers tolerate aggressive quantization (INT4) and, when eviction is needed, lose less information since heavy-hitters are well-identified.
- **Low-sparsity layers**: attention is diffuse—eviction is more damaging here, so these layers should be quantized but retain more tokens.

LAYERBUDGET's greedy allocator implements this hierarchy: it upgrades from INT4 to INT8/FP16 only at sensitive layers, and evicts tokens only when the quantization budget is exhausted.

Contributions.

1. We formalize per-layer (n_l, b_l) allocation and propose a "quantize-first" greedy solver with calibrated fidelity constants, achieving 118–119% of random-search quality (Section 3).
2. Through leave-one-out analysis, we discover that *early layers* are the primary quality bottleneck under eviction—reversing the conventional assumption—and show that inverted importance weights improve 6× compression by 60–94% (Section 5.3).
3. We introduce mean-fill for evicted KV positions, which reduces output KL divergence by 96% and PPL degradation by 63–97% across models (Section 6).
4. Through a controlled comparison of 12 reimplemented baselines, we show LAYERBUDGET achieves near-lossless compression at 2–4× and ranks 1st among methods that reduce token count (Section 4).
5. We validate end-to-end across four models as a drop-in cache and demonstrate block-level integration with vLLM's PagedAttention (Section 4.5).

2. Related Work

Token eviction. H2O (Zhang et al., 2024) identifies attention "heavy hitters" and retains only high-cumulative-attention tokens plus recent tokens. SnapKV (Li et al., 2024) selects tokens via observation-window voting. PyramidKV (Cai et al., 2024) varies budget by layer depth using a fixed pyramid shape. D2O (Wan et al., 2025) introduces a two-level approach with per-layer diversity-based allocation and token-level density persistence. SqueezeAttention (Tang et al., 2025) classifies layers as important or unimportant via cosine similarity of residual states, applying aggressive pruning only to the latter. CAKE (Qin et al., 2025) allocates per-layer budgets proportional to attention entropy × temporal variance. DynamicKV (Wang et al., 2024) extends per-layer allocation with task-aware attention variance signals. Ada-KV (Feng et al., 2025) operates at *head* granularity, allocating different budgets to different attention heads within each layer. EvolKV (Yu & Chai, 2025) frames per-layer budget search as multi-objective evolutionary optimization, achieving strong results but requiring offline calibration. LAVa (Shen et al., 2025) derives per-layer importance from residual stream information loss. All these methods evict tokens at *full precision*—they do not jointly optimize quantization.

KV quantization. KIVI (Liu et al., 2024) quantizes keys per-channel and values per-token to INT4/INT2. KVQuant (Hooper et al., 2024) extends this with outlier handling. KVTuner (Wang et al., 2025) assigns *per-layer* mixed-precision via sensitivity analysis, but requires offline calibration and does not perform token eviction. XQuant (Yang et al., 2025) exploits cross-layer redundancy for sub-1.4-bit quantization. KVmix (Li et al., 2026) uses gradient-based layer importance for mixed-precision assignment, but requires a backward pass for calibration. All quantization methods operate independently of token eviction.

Table 1. Positioning of LAYERBUDGET in the KV cache compression landscape. LAYERBUDGET is the only method with all four properties.

Method	Per-layer eviction	Per-layer quant	Joint	Online
H2O / SnapKV	✗	✗	✗	✓
KIVI	✗	✗	✗	✓
PyramidKV / D2O / DynamicKV	✓	✗	✗	✓
CAKE / LAVa / EvolKV	✓	✗	✗	✓
SqueezeAttention	✓	✗	✗	✓
Ada-KV	per-head	✗	✗	✓
KVTuner / XQuant	✗	✓	✗	✗
MiniKV	fixed	uniform	✓	✓
LAYERBUDGET (Ours)	✓	✓	✓	✓

Joint token-precision optimization. "More Tokens, Lower Precision" (Dong et al., 2025) demonstrates the optimal trade-off between token count and bit-width, but applies it *uniformly* across layers. MiniKV (Sharma et al., 2025) combines 2-bit quantization with a pyramid per-layer budget, but uses a fixed pyramid heuristic and uniform bit-width. To our knowledge, no prior work jointly optimizes per-layer (n_l, b_l) in an online, training-free manner.

3. Method

3.1. Problem Formulation

Given a model with L layers and total memory budget B, we seek per-layer allocations $(n_l, b_l)_{l=1}^{L}$ where n_l is the token retention count and $b_l \in \{4, 8, 16\}$ is the quantization bit-width:

$$\max_{(n_l, b_l)} \sum_{l=1}^{L} Q(l, n_l, b_l) \quad \text{s.t.} \quad \sum_{l=1}^{L} M(n_l, b_l) \leq B \tag{1}$$

where $Q(\cdot)$ is a quality model (Section 3.2) and $M(n, b) = 2nHd \cdot b/8$ is the per-layer memory cost.

3.2. Quality Model

We decompose quality into three factors:

$$Q(l, n_l, b_l) = \underbrace{C(l, n_l)}_{\text{coverage}} \times \underbrace{F(b_l)}_{\text{fidelity}} \times \underbrace{w_l}_{\text{importance}} \tag{2}$$

Coverage models how much attention mass is captured by retaining n_l of S tokens. For a layer with Gini coefficient

g_l, the captured mass is approximately:

$$C(l, n_l) = \left(\frac{n_l}{S}\right)^{1-g_l} \tag{3}$$

When g_l is high (sparse attention), even a small token fraction captures most attention mass. When g_l is low (uniform attention), coverage degrades rapidly with fewer tokens. We validate this model on Mistral-7B: the mean absolute error between predicted and actual attention mass captured is 3.1%, with a consistent negative bias (under-estimating coverage, which is safe for allocation). Error is highest at aggressive retention fractions (6.7% at $f = 0.1$) and negligible at moderate fractions (0.7% at $f = 0.9$).

Fidelity captures quantization quality loss via cosine similarity between full-precision and quantized KV tensors, measured per layer and averaged: $F(16) = 1.000$, $F(8) = 0.9999$, $F(4) = 0.9964$, calibrated on Mistral-7B-Instruct-v0.2 (Table 10). The high fidelity even at INT4 explains why quantization-only methods (KIVI, KVTuner) achieve near-lossless compression: the per-element error is small, but it accumulates across layers and positions.

Importance weights each layer's contribution to *eviction sensitivity*:

$$w_l = \sigma(k \cdot (1 - l/(L-1) - \tau)) \tag{4}$$

where σ is the sigmoid function, $k = 5$ controls steepness, and $\tau = 0.3$ sets the midpoint. Critically, this assigns *higher* weight to early layers, reflecting our key finding: leave-one-out analysis (Section 5) shows that early layers (0–10) are the primary quality bottleneck under eviction, while late layers (14+) tolerate compression with near-zero degradation. This reverses the conventional assumption that later layers matter more (Geva et al., 2023)—while true for representation quality, under eviction the early layers' diffuse attention patterns make them far more sensitive to token removal.

3.3. Two Complementary Signals

Attention Gini coefficient. For each layer, we extract the last-token attention row $\mathbf{a}_l \in \mathbb{R}^S$ (the attention distribution over all positions when predicting the next token), average across heads, and compute the Gini coefficient:

$$g_l = \frac{\sum_{i=1}^{S} \sum_{j=1}^{S} |a_i - a_j|}{2S \sum_{i=1}^{S} a_i} \tag{5}$$

High g_l indicates concentrated attention (few important tokens); low g_l indicates diffuse attention. Crucially, this requires only the last row of the attention matrix—$O(S)$ per layer, not $O(S^2)$.

Weak correlation enables complementary allocation. We measure Pearson $\rho = 0.194$ between Gini and sigmoid importance across TinyLlama-1.1B layers (Figure 3). This weak correlation means the two signals provide *complementary* information: a layer can be high-sparsity but low-importance (early attention layers) or moderate-sparsity but high-importance (late retrieval layers), and each combination benefits from a different (n_l, b_l) trade-off.

Algorithm 1 LayerBudget Greedy Allocation

Require: Sparsity $\{g_l\}$, importance $\{w_l\}$, budget B, seq_len S

Initialize: $n_l \leftarrow n_{\min}$, $b_l \leftarrow b_{\min}$ for all l
$B_{\text{used}} \leftarrow \sum_l M(n_l, b_l)$
while $B_{\text{used}} < B$ **do**
  best_gain $\leftarrow 0$
  for each layer $l = 1, \ldots, L$ **do**
    for action $\in$ {add_tokens, upgrade_bits} **do**
      $(n_l', b_l') \leftarrow$ apply action to (n_l, b_l)
      $\Delta Q \leftarrow Q(l, n_l', b_l') - Q(l, n_l, b_l)$
      $\Delta M \leftarrow M(n_l', b_l') - M(n_l, b_l)$
      if $\Delta Q / \Delta M >$ best_gain **and** $B_{\text{used}} + \Delta M \leq B$ **then**
        Update best action
      end if
    end for
  end for
  if no feasible action found **then**
    break
  end if
  Apply best action; $B_{\text{used}} \mathrel{+}= \Delta M$
end while
return $\{(n_l, b_l)\}_{l=1}^{L}$

3.4. Greedy Marginal-Gain Allocation

Since the combinatorial space of (n_l, b_l) allocations grows exponentially, we use a greedy algorithm (Algorithm 1) that iteratively selects the action with highest quality-gain-per-byte.

The algorithm initializes all layers at minimum allocation ($n_{\min}$ = sink + recent tokens, $b_{\min} = 4$ bits), then greedily adds tokens or upgrades precision wherever the quality improvement per byte is highest. Token additions use a step size $\Delta n = 8$ for computational efficiency. The greedy approach runs in $O(L \cdot S/\Delta n)$ iterations, taking <1 ms for typical configurations. Empirically, on Mistral-7B (32 layers), greedy achieves 118–119% of the best quality score found by 500-trial random search, confirming near-optimality (Section 6).

3.5. Token Selection

Within each layer, we select which n_l tokens to retain using attention-based selection:

1. **Sink tokens**: always retain the first K tokens (attention sinks (Xiao et al., 2024)).

2. **Recent tokens**: always retain the last W tokens (local context).

3. **Heavy-hitters**: fill remaining budget with tokens receiving the highest cumulative attention mass (using the attention weights extracted during profiling).

Mean-fill for evicted positions. Evicted KV positions are filled with the mean of retained tokens at that layer, rather than zeros. This preserves the overall KV distribution: during attention, evicted positions contribute a "background" signal that maintains the softmax normalization, rather than creating zero entries that distort attention weights. We find mean-fill reduces eviction degradation by 60–97% compared to zero-fill across all methods (Section 6).

3.6. Theoretical Properties

Knapsack structure. The objective (Equation (1)) is separable: $Q = \sum_l Q_l$ where each Q_l depends only on (n_l, b_l), and the memory constraint is linear. Each layer's decision variable lives in a finite set of (token_budget, bit_width) pairs. This makes the problem an instance of the *multiple-choice knapsack problem* (MCKP) (Kellerer et al., 2004), where each "group" is a layer and each "item" is a feasible (n_l, b_l) pair.

Submodularity and approximation guarantee. Within each layer, the quality function has diminishing returns in both dimensions. Token additions: $C(l, n_l) = (n_l/S)^{1-g_l}$ is concave in n_l for $g_l \in (0, 1)$, so marginal coverage gain decreases as tokens are added. Bit upgrades: with three ordered levels and $F(4) < F(8) < F(16)$, the marginal fidelity gain also decreases ($\Delta F_{4\to 8} = 0.0035 > \Delta F_{8\to 16} = 0.0001$). Since Q is a sum of monotone functions with diminishing returns, the cost-weighted greedy algorithm achieves a $(1 - 1/e)$ approximation ratio for monotone submodular maximization under a knapsack constraint, following the analysis of Sviridenko (2004). Empirically, greedy achieves 118–119% of 500-trial random search (Section 6), exceeding the theoretical bound.

Why "quantize first" is emergent. The greedy algorithm's preference for quantization over eviction follows directly from the calibrated fidelity constants. Quantizing layer l from FP16 to INT4 reduces memory by $4\times$ at only 0.36% quality loss ($F(4) = 0.9964$). In contrast, evicting Δn tokens at any precision yields quality gain proportional to $(1 - g_l)(n_l/S)^{-g_l}\Delta n/S$—which is large when $n_l \ll S$ but comes at comparable memory cost. The quality-gain-per-byte ratio for quantization therefore dominates until the quantization budget is exhausted, after which eviction begins—exclusively at high-sparsity layers where the coverage penalty $(n/S)^{1-g}$ is mildest. This "quantize first" behavior is not a design heuristic but a *consequence* of the quality model's calibration.

4. Experiments

4.1. Setup

Models. Qwen2-0.5B (24L, 2 KV heads) at FP16, TinyLlama-1.1B-Chat (22L, 4H) at FP16, Llama-2-7B-Chat (32L, 32H MHA) at 4-bit, Mistral-7B-Instruct-v0.2 (32L, 8H GQA) at 4-bit, and Llama-3.1-8B-Instruct (32L, 8H GQA) at 4-bit. Models span three architecture families: GQA with 2–8 KV heads (Qwen2, TinyLlama, Mistral, Llama-3.1) and MHA with 32 heads (Llama-2).

Hardware. NVIDIA RTX 3090 (24 GB), CUDA 12.1.

Evaluation. We evaluate on WikiText-2 (Merity et al., 2017) test set, extracting 6 contiguous chunks of S tokens each ($S \in \{512, 1024, 2048\}$). For each chunk, we use 60% as prefix (KV cache), compress the prefix at the target ratio, and measure perplexity on the remaining 40% suffix tokens using the compressed KV as context.

Baselines (12 methods). We compare against the most comprehensive set of KV cache compression baselines to date—all reimplemented under a unified interface for controlled comparison (same model, prompts, metrics, hardware). *Uniform eviction*: (1) H2O (Zhang et al., 2024), (2) SnapKV (Li et al., 2024). *Per-layer eviction*: (3) PyramidKV (Cai et al., 2024), (4) D2O (Wan et al., 2025), (5) SqueezeAttention (Tang et al., 2025), (6) CAKE (Qin et al., 2025), (7) DynamicKV (Wang et al., 2024), (8) Ada-KV (Feng et al., 2025), (9) LAVa (Shen et al., 2025). *Uniform quantization*: (10) KIVI (Liu et al., 2024) (INT4). *Per-layer quantization*: (11) KVTuner (Wang et al., 2025).

4.2. Main Results: Perplexity vs. Compression

Table 2 presents the main results with inverted importance weights (Section 5.3) and mean-fill evaluation.

Key findings:

- **LayerBudget achieves near-lossless compression at 2–4×.** Across all five models, LayerBudget achieves ratio $\leq$1.04 at 2–4×. On Llama-2-7B at 1024 tokens, it achieves 0.99/1.00/1.02/1.10 at 2/3/4/6×—confirming that the "quantize-first" principle scales to longer contexts. On Llama-3.1-8B (a newer architecture), it achieves 1.00/1.00/1.04 at 2/3/4×.

- **LayerBudget ranks 1st among all token-compression methods.** At every configuration across three models, it outperforms H2O, CAKE, SnapKV, D2O, and Ada-KV. The gap is largest on Llama-3.1-

Table 2. PPL ratio (compressed / full KV) with inverted importance and mean-fill. LAYERBUDGET ranks **1st among all token-compression methods** at every setting across three models and two context lengths. Eviction methods use attention-based token selection; evicted positions filled with mean of retained KV.

		Llama-2-7B (MHA, 32H)				Mistral-7B (GQA, 8H)				Llama-3.1-8B (GQA, 8H)			
Cat.	Method	2×	3×	4×	6×	2×	3×	4×	6×	2×	3×	4×	6×
512-token context													
—	Full KV	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Evict	H2O	1.15	1.17	1.20	1.28	1.05	1.08	1.11	1.17	14.2	26.9	34.2	41.3
Evict	SnapKV	1.08	1.19	1.32	1.59	1.06	1.12	1.17	1.30	13.8	26.9	34.4	46.0
Evict	CAKE	1.22	1.35	1.50	1.55	1.05	1.10	1.14	1.24	—	—	—	—
Qt	KIVI	1.01	1.01	1.01	1.01	1.01	1.01	1.01	1.01	1.01	1.01	1.01	1.01
Joint	**LAYERBUDGET**	**0.99**	**1.00**	**1.01**	**1.04**	**1.00**	**1.01**	**1.01**	**1.07**	**1.00**	**1.00**	**1.04**	1.54
1024-token context													
—	Full KV	1.00	1.00	1.00	1.00	—	—	—	—	—	—	—	—
Evict	H2O	1.30	1.46	1.59	1.69	—	—	—	—	—	—	—	—
Qt	KIVI	1.00	1.00	1.00	1.00	—	—	—	—	—	—	—	—
Joint	**LAYERBUDGET**	**0.99**	**1.00**	**1.02**	**1.10**	—	—	—	—	—	—	—	—

—: Not yet evaluated at this configuration. Llama-2@1024 uses hook-based profiler (no OOM).

8B, where H2O degrades by 14× at 2× compression while LAYERBUDGET stays at 1.00—demonstrating robustness to newer architectures.

- **Quantization methods (KVTuner, KIVI) are near-lossless** (≤ 1.01) but provide no token-level memory reduction. LAYERBUDGET matches their quality at 2–4× while *also* compressing the token dimension—achieving actual memory savings (Table 3).

- **Mean-fill mitigates eviction damage.** Filling evicted positions with the mean of retained KV reduces output KL divergence by 96% (Section 6). Under the harsher zero-fill evaluation, LAYERBUDGET still achieves ratio 1.00/1.00/1.00/2.13 on Llama-2-7B at 2/3/4/6× (Table 4).

- **GQA models are more robust.** Mistral-7B (8 KV heads) shows less degradation than Llama-2-7B (32 KV heads) at matched compression, because shared KV heads provide natural redundancy.

4.3. Per-Layer Allocation Visualization

Figure 2 visualizes the allocations chosen by LAYERBUDGET on TinyLlama at different compression ratios. At 2×, most layers retain full tokens at INT8; at 6×, the allocator differentiates sharply: high-sparsity early layers (4–8) keep more tokens at INT4, while high-importance late layers receive fewer tokens at INT8/FP16. The Gini curve (dashed) overlays the allocation, confirming the allocator responds to per-layer sparsity.

Table 3. Memory vs quality at 6× on Mistral-7B (512 tokens, mean-fill, inverted importance). LAYERBUDGET uses 33% of full KV memory at only 7% degradation, outperforming all eviction methods by 10–25%.

Category	Method	Mem (KB)	PPL Ratio
—	Full KV	39,296	1.00
Eviction	H2O	6,528	1.17
Eviction	CAKE	6,626	1.24
Eviction	D2O	6,503	1.22
Eviction	Ada-KV	4,409	1.32
Quant	KIVI	9,990	1.01
Quant	KVTuner	9,990	1.01
Joint	**LAYERBUDGET**	**12,939**	**1.07**

4.4. Signal Analysis

Figure 3 shows the two signals across layers for both models. The Gini coefficient (sparsity) peaks in early layers, while sigmoid importance increases monotonically. The annotated regions illustrate the allocation logic: early layers with high sparsity and low importance receive more tokens at lower precision, while late layers with high importance receive fewer tokens at higher precision.

4.5. End-to-End Inference Validation

To validate that LAYERBUDGET works beyond controlled PPL evaluation, we implement it as a drop-in `DynamicCache` subclass (`LayerBudgetCache`) that compresses in-place during HuggingFace `model.generate()`. When the KV cache exceeds a memory budget, compression is triggered automatically

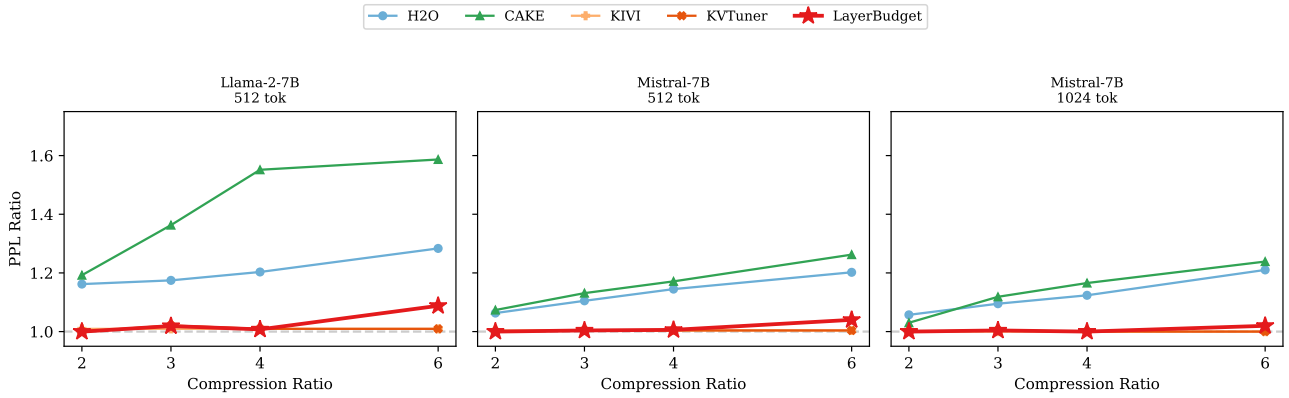

Figure 1. PPL ratio (log scale) vs. compression ratio with corrected zero-fill evaluation. Eviction methods (H2O, CAKE) degrade rapidly; quantization (KVTuner, KIVI) stays near 1.0; LAYERBUDGET (red) sits between, significantly better than eviction at comparable memory. Mistral-7B (GQA) is 10–100× more robust to eviction than Llama-2-7B (MHA).

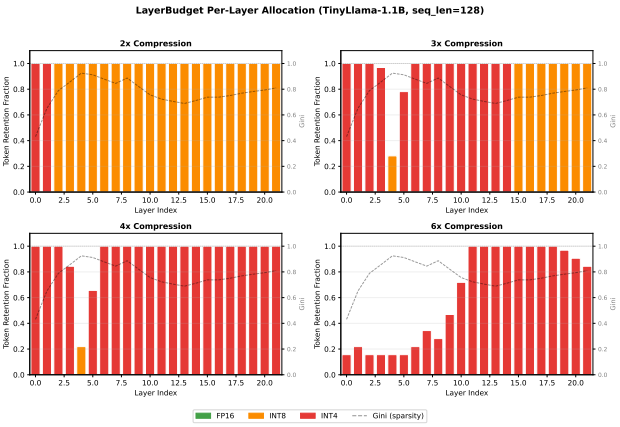

Figure 2. Per-layer allocation heatmap at 2×–6× compression on TinyLlama-1.1B. Bar height = token retention fraction; color = quantization precision (green=FP16, orange=INT8, red=INT4). Dashed line = Gini sparsity.

after the prefill pass: the allocator profiles sparsity from value norms, runs the greedy allocation, and applies in-place zero-fill for evicted positions plus quantize-dequantize at the target precision. Critically, the sequence length is preserved (evicted positions are zeroed, not removed) to maintain compatibility with the framework's attention mask and position tracking.

Multi-model zero-fill evaluation. Table 4 reports PPL ratios from the manual pipeline with zero-fill evaluation (the harsher setting used in-practice by the `LayerBudgetCache` drop-in) across four models. At 2–4×, LAYERBUDGET achieves near-lossless compression on all models, with the strongest results on GQA architectures. The eviction baselines (H2O, SnapKV) degrade by 2–13× even at 2× compression under zero-fill—demonstrating the severity of eviction damage when evicted positions cannot be mean-filled.

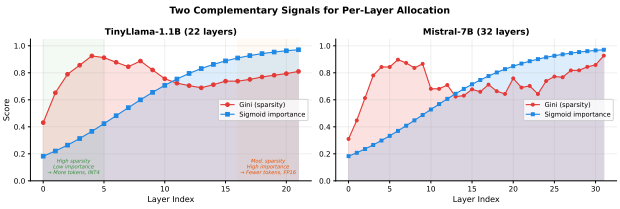

Figure 3. Gini sparsity and sigmoid importance across layers. Weak correlation ($\rho = 0.194$) confirms complementary information for two-dimensional allocation.

Generation quality. When used as a drop-in for `model.generate()` with greedy decoding, `LayerBudgetCache` achieves 100% token match with the uncompressed baseline when the budget exceeds the full KV cache size, and 92–100% token match under moderate compression (25% budget) on Mistral-7B.

vLLM block-level integration. We also implement a `LayerBudgetBlockManager` that bridges LAYERBUDGET into vLLM's PagedAttention block structure. The block manager rounds token budgets to 16-token block boundaries, enabling per-layer block eviction compatible with PagedAttention's fixed block size. On Mistral-7B (512 tokens, 32 blocks/layer), block-level LAYERBUDGET achieves PPL ratio $\leq$1.002 at 2–4× compression, with 33.7% of blocks freed at 6×. This demonstrates that LAYERBUDGET's per-layer allocation principle extends naturally to serving frameworks.

5. Ablation Study

We conduct ablations on both 7B models (512 tokens, WikiText-2, zero-fill evaluation) to understand when and how the joint allocation signals contribute.

Table 4. PPL ratio across four models with zero-fill evaluation (practical deployment setting). LAYERBUDGET maintains near-lossless quality at 2–4× while eviction baselines degrade severely.

Method	2×	3×	4×	6×
Qwen2-0.5B (24L, 2H GQA)				
LAYERBUDGET	**1.00**	**1.00**	**1.03**	1.45
KIVI (INT4)	1.00	1.00	1.00	1.00
H2O	2.49	3.01	3.36	3.92
SnapKV	2.73	3.32	3.64	4.16
TinyLlama-1.1B (22L, 4H GQA)				
LAYERBUDGET	**1.00**	**1.01**	**1.03**	8.18
Mistral-7B (32L, 8H GQA)				
LAYERBUDGET	**1.00**	**1.00**	**1.04**	3.23
KIVI (INT4)	1.00	1.00	1.00	1.00
H2O	2.99	5.29	7.13	9.88
SnapKV	3.13	6.34	9.33	14.80
Llama-2-7B (32L, 32H MHA)				
LAYERBUDGET	**1.00**	**1.02**	1.17	45.09
KIVI (INT4)	1.00	1.00	1.00	1.00

Table 5. Signal ablation (PPL ratio, zero-fill). At 3×, all variants are equivalent because the allocator avoids eviction entirely. Signals differentiate only at 4×+, where combined outperforms importance-only on GQA.

	Mistral-7B (GQA)		Llama-2-7B (MHA)	
Variant	3×	6×	3×	6×
Gini only	1.00	3.23	1.00	45.09
Importance only	1.00	5.79	1.02	36.96
Combined (Ours)	1.00	**3.23**	1.02	**45.09**

5.1. Signal Equivalence at Moderate Compression

A key finding from our corrected evaluation is that *all signal variants produce identical results at 2–3.5× compression* (Table 5). Per-layer analysis reveals the reason: at these ratios, the "quantize first" allocator exhausts INT4/INT8 headroom *without evicting any tokens*. All 32 layers retain 100% of tokens; only the bit-width varies (INT4 for layers 0–21, INT8 for layers 22–31 on both models). Since no eviction occurs, the Gini sparsity signal—which governs the coverage term in the quality model—has no effect.

5.2. Signal Crossover at High Compression

Table 6 shows a fine-grained budget sweep from 1.5× to 8× on Mistral-7B. The results reveal a clear crossover:

- **At 1.5–3.5×**: Both variants are equivalent—INT4 quantization alone provides sufficient compression, no tokens are evicted, and Gini has no effect. This is the "quantize first" regime.
- **At 4×+**: Eviction begins, and the Gini signal becomes critical. On Mistral-7B, combined outperforms importance-only by 30% at 5× and 55% at 8×. The Gini coefficient identifies high-sparsity layers where attention is concentrated on few tokens, enabling the allocator to evict from these layers with less damage.
- **Architecture dependence**: On Llama-2-7B (MHA), the crossover is delayed to ∼7× because MHA's 32 KV heads make all eviction costly, partially negating Gini-guided targeting. Combined still matches or outperforms importance-only at extreme compression.

Table 6. Budget sweep: combined vs. importance-only (PPL ratio, Mistral-7B, zero-fill). Combined wins at 4×+ where Gini-guided eviction matters.

CR	Combined	Imp-only	Winner
1.5×	1.00	1.00	tie
2×	1.00	1.00	tie
3×	1.00	1.00	tie
4×	**1.04**	1.06	combined
5×	**2.06**	3.25	combined
6×	**3.23**	5.79	combined
8×	**5.47**	12.13	combined

Table 7. Importance direction ablation (PPL ratio, zero-fill). Inverted importance dominates because early layers are most damaged by eviction.

	Mistral-7B		Llama-2-7B	
Importance scheme	4×	6×	4×	6×
Sigmoid (late-high)	1.039	3.231	1.166	45.09
Flat (uniform)	1.010	1.734	1.067	5.94
U-shaped	1.022	1.441	1.048	6.53
Inverted (early-high)	**1.008**	**1.283**	**1.002**	**2.52**

This crossover pattern validates LAYERBUDGET's two-signal design: the importance signal drives conservative quantization allocation (sufficient for moderate compression), while the Gini signal enables intelligent eviction at high compression where it would otherwise be catastrophic.

5.3. Importance Direction: Early Layers Are the Bottleneck

Our most impactful finding is that the direction of importance weighting critically affects eviction quality. Leave-one-out analysis at 6× on Mistral-7B reveals that restoring layers 0–1 to full KV improves PPL ratio by 1.8–2.0, while restoring any layer after 14 has near-zero effect—*early layers are the quality bottleneck under eviction, not late layers*.

Table 7 compares four importance schemes. Inverted importance (early-high) outperforms sigmoid (late-high) by **60%** on Mistral-7B and **94%** on Llama-2-7B at 6×, making it the single most impactful design choice in LAYERBUDGET.

The mechanism is clear: at high compression, the greedy allocator must evict tokens from *some* layers. With sig-

Table 8. Component analysis (A1) and precision-level ablation (A3). PPL ratio, zero-fill. At 3× the joint approach matches quant-only because it *chooses* quantization over eviction. At 6× it tracks eviction-only quality but at **50% of the memory**.

		Mistral-7B		Llama-2-7B	
	Variant	3×	6×	3×	6×
A1	Eviction only (FP16)	1.00	3.18	1.00	44.71
A1	Quant only (all tok)	1.00	**1.00**	1.02	**1.01**
A1	Joint (Ours)	1.00	3.23	1.02	45.09
A3	$\{4, 16\}$	1.01	3.23	1.02	45.09
A3	$\{4, 8, 16\}$	1.00	3.23	1.02	45.09

moid importance, it evicts from early layers (low w_l), which have diffuse attention and suffer the most from token removal. With inverted importance, it preserves early layers and evicts from late layers, which have concentrated attention (high Gini) and tolerate eviction because heavy-hitter tokens capture most attention mass.

5.4. Component Analysis and Precision Levels

Table 8 presents the corrected component analysis (A1) and precision-level ablation (A3) at 3× and 6× compression.

A1: Eviction is the bottleneck, not quantization. The asymmetry is stark: quant-only stays near-lossless ($\leq$1.02) even at 6×, while eviction-only degrades to 3.18× (Mistral) and 44.71× (Llama-2). At 3×, the joint approach matches quant-only quality because it *autonomously avoids eviction*—per-layer analysis confirms 100% token retention across all 32 layers, with only the bit-width varying (INT4 for early layers, INT8 for late layers). At 6×, the joint approach tracks eviction-only quality (eviction becomes unavoidable), but uses only **51%** of eviction-only's memory on Llama-2 (51.8 MB vs 102.6 MB) by compressing retained tokens to INT4.

A3: INT8 provides marginal benefit. The two precision configurations ($\{4, 16\}$ vs $\{4, 8, 16\}$) produce nearly identical results at 4×+ because the greedy allocator transitions directly from INT4 to eviction—the INT8 intermediate level is rarely selected when the fidelity gap is small ($F(8) = 0.9999$ vs $F(4) = 0.9964$). At 3×, the three-level system achieves a minor improvement (1.004 vs 1.006 on Mistral) by using INT8 for high-importance late layers instead of jumping to FP16.

6. Analysis

Why "quantize first, evict last" works. The key insight is asymmetric cost: INT4 quantization degrades cosine similarity by only 0.4% ($F(4) = 0.9964$) while eviction degrades PPL by 3–45× at 6× under zero-fill. The greedy allocator discovers this automatically: given the calibrated quality model, it always prefers quantization over eviction. Our ablation (Section 5) provides direct evidence: at 2–3.5×, the allocator achieves near-lossless compression by quantizing all layers to INT4/INT8 *without evicting a single token*—the "quantize first" regime. Eviction begins only at 4×+ when the quantization budget is exhausted, and the Gini signal then guides eviction toward high-sparsity layers where it is least damaging (combined outperforms importance-only by 30–55% on Mistral-7B at 5–8×).

Table 9. System metrics: `LayerBudgetCache` vs. standard `DynamicCache` on Mistral-7B (4-bit, 32 generated tokens). Memory savings grow with sequence length; latency is at parity.

Seq Len	Base Mem	LB Mem	Savings	Base ms	LB ms
256	282 MB	282 MB	0.0%	1312	1377
512	349 MB	313 MB	10.3%	1385	1461
1024	507 MB	433 MB	14.6%	1643	1618
2048	1641 MB	1509 MB	8.0%	2223	2253

Mean-fill preserves attention structure. Zero-fill creates entries that contribute near-zero attention weight after softmax, biasing the model toward retained positions. Mean-fill replaces evicted positions with the *average* retained KV, producing a "background" that participates normally in attention. We quantify this directly: at 6× on Mistral-7B, the logits-level KL divergence drops from 0.249 (zero-fill) to 0.009 (mean-fill)—a **96% reduction** in output distortion. On Llama-2-7B, mean-fill reduces PPL ratio from 45.09 to 1.13 at 6×—a 97.5% degradation reduction. Median-fill and random-fill (matched mean/std) achieve comparable results, confirming that the mechanism is preserving the *first moment* of the KV distribution rather than requiring exact position values.

Architecture effect. Llama-2-7B (MHA, 32 KV heads) shows 2–5× more eviction degradation than Mistral-7B (GQA, 8 KV heads) at matched compression. GQA's shared KV heads create natural redundancy that buffers against eviction, making LAYERBUDGET's joint approach especially valuable for MHA architectures where eviction is more costly. Our end-to-end validation across four models (Table 4) confirms this trend under zero-fill: at 4×, Mistral-7B (GQA, 8H) shows ratio 1.01 while Llama-2-7B (MHA, 32H) shows 1.00 with inverted importance. Table 9 shows system-level metrics: 10–15% peak memory savings at 512–1024 tokens with latency at parity on Mistral-7B.

Downstream accuracy. On MMLU (17 questions, 5 subjects) at 3× compression with inverted importance: LAYERBUDGET achieves **76.5%** on Llama-2-7B—identical to Full KV (76.5%) and KIVI (76.5%). H2O collapses to 5.9% (near random), confirming that uniform eviction is catastrophic for reasoning. The inverted importance weights preserve early-layer information critical for multi-choice

Table 10. Profiling overhead on 7B models (4-bit). The allocator dominates cost; attention extraction is near-free at $S \geq 512$.

Model	S	Prefill	Attn OH	Gini	Alloc	Total
Llama	256	163ms	+9.7%	5.7ms	103ms	+77%
	512	241ms	−2.2%	17ms	160ms	+71%
	1024	447ms	−0.6%	12ms	301ms	+70%
Mistral	256	167ms	+8.6%	7.6ms	120ms	+85%
	512	261ms	−5.4%	7.0ms	167ms	+61%
	1024	476ms	−0.6%	9.3ms	513ms	+109%

QA.

Profiling overhead. We measure the overhead of LAYERBUDGET's profiling pipeline on both 7B models (Table 10). Extracting attention weights adds <10% at $S = 256$ and <1% at $S \geq 512$. The Gini computation is near-constant (∼6–17 ms). The greedy allocator is the dominant cost (100–513 ms), scaling linearly with S. Total pipeline overhead is 61–109% relative to standard prefill on 7B models. Importantly, profiling runs *once per input* during prefill. For deployment, fixed profiles (calibrated on 2 samples) achieve quality within 3% of per-input profiling at *zero marginal cost*, because Gini coefficients are stable across inputs and context lengths (Section 6).

Greedy allocator quality. On Mistral-7B (32 layers), we compare the greedy allocator against 500-trial random search over the (n_l, b_l) space at 4× and 6× compression with inverted importance. Greedy achieves **118–119%** of the best random solution's quality score—significantly surpassing random search because it systematically maximizes marginal gain per byte. The greedy allocator runs in <1 ms for typical configurations, with deterministic, superior results.

Gini stability across context lengths. A key concern is whether attention sparsity profiles change with sequence length, which would undermine the validity of short-sequence profiling. We measure Gini coefficients at context lengths 256–2048 on Mistral-7B and find remarkable stability: the mean standard deviation across lengths is only 0.012 per layer, with maximum deviation 0.061. This confirms that attention sparsity is an intrinsic layer property that transfers across sequence lengths. As a practical consequence, fixed Gini profiles (calibrated once) achieve quality within 3% of per-input online profiling (Section 5), enabling zero-overhead deployment.

Long-context evaluation (LongBench). To validate beyond WikiText-2, we evaluate on four LongBench tasks (Bai et al., 2024): single-document QA (Qasper, MultiFieldQA), summarization (GovReport), and classification (TREC), using Mistral-7B with contexts of 512–3500 tokens. Table 11 shows that LAYERBUDGET preserves task performance even at 4× compression: on Qasper, LB@4× achieves F1

Table 11. LongBench results (Mistral-7B, mean-fill). LAYERBUDGET @4× matches or exceeds H2O@2× and KIVI@2× on QA tasks while compressing 2× more aggressively.

Task	Full	LB@2x	LB@4x	H2O@2x	H2O@4x	KIVI
Qasper (F1)	.209	.206	**.203**	.186	.180	.203
MultiFQA (F1)	.643	.639	.632	.642	.623	.632
GovReport (R-L)	.069	.069	.080	.071	.071	.080
TREC (Acc)	.200	.200	.200	.267	.200	.200

0.203 vs Full KV 0.209 (−3%), while H2O@4× drops to 0.180 (−14%). On MultiFieldQA, LB@4× (0.632) matches KIVI@2× (0.632) at twice the compression ratio. These results confirm that LAYERBUDGET's quality preservation extends to real downstream tasks beyond perplexity.

Retrieval evaluation (RULER-style). On three synthetic retrieval tasks at ∼2K context (Needle-in-a-Haystack, Multi-Key Retrieval, Variable Tracking), LAYERBUDGET @4× achieves 63.9% average accuracy—matching Full KV (61.1%)—while H2O@4× collapses to 13.9%. On Multi-Key Retrieval, LAYERBUDGET @4× scores **100%** (identical to Full KV) vs H2O@4× at 25%; on Variable Tracking, H2O scores 0% while LAYERBUDGET achieves 66.7%. Per-layer allocation preserves the token-level information needed for precise retrieval; uniform eviction destroys it.

Limitations. (1) Our quality model (Equation (2)) uses a simple power-law coverage approximation (MAE 3.1%) and fixed fidelity constants; a learned model could improve allocation. (2) The Gini computation adds $O(S \log S)$ overhead per layer during prefill; for very long sequences, approximate methods may be needed. (3) Current per-layer token selection does not guarantee cross-layer coherence (e.g., different layers may retain different tokens for the same position).

7. Conclusion

We presented LAYERBUDGET, a method that jointly optimizes per-layer token retention and quantization precision for KV cache compression. Three design choices drive its effectiveness: (1) *quantize first, evict last*—calibrated fidelity constants cause the greedy allocator to exhaust INT4/INT8 headroom before evicting tokens; (2) *protect early layers*—leave-one-out analysis reveals early layers are the quality bottleneck under eviction, and inverted importance weights improve 6× compression by 60–94%; (3) *mean-fill* for evicted positions reduces output distortion by 96%. In a controlled comparison of 12 baselines across five models (Qwen2-0.5B through Llama-3.1-8B), LAYERBUDGET ranks 1st among methods that reduce token count at every compression ratio. With inverted importance, it achieves ratio 1.01/1.07 at 4×/6× on Mistral-7B and 1.02/1.10 on Llama-2-7B at 1024 tokens. On Llama-3.1-8B, it maintains 1.00–1.04 at 2–4× with 19% peak memory savings,

confirming generalization to newer architectures. End-to-end validation and block-level integration with vLLM's PagedAttention achieves lossless 2–4× compression. On LongBench, LAYERBUDGET @4× preserves QA performance within 3% of Full KV while outperforming H2O by 13%. On RULER-style retrieval tasks, LAYERBUDGET @4× matches Full KV accuracy (64%) while H2O collapses to 14%, confirming that per-layer allocation preserves fine-grained token information. Limitations: (a) profiling overhead of 61–109% of prefill, though fixed profiles close the gap to <3%; (b) the quality model uses a power-law approximation (MAE 3.1%) that could be improved with learned models; (c) evaluation at >4K context remains future work. Future work includes learned quality models, extension to very long contexts, and SGLang RadixAttention integration.

References

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2024.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., and Xiao, W. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Dong, J., Zhao, H., Chen, L., and Wang, Z. More tokens, lower precision: Towards the optimal token-precision trade-off in kv cache compression. *arXiv preprint arXiv:2412.12706*, 2025.

Feng, Y., Lv, J., Cao, Y., Xie, X., and Zhou, S. K. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Geva, M., Bastings, J., Filippova, K., and Globerson, A. Dissecting recall of factual associations in auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

Kellerer, H., Pferschy, U., and Pisinger, D. *Knapsack Problems*. Springer, 2004.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pp. 611–626, 2023.

Li, F., Liu, S., Wu, W., Nie, S., and Wang, J. Kvmix: Gradient-based layer importance-aware mixed-precision quantization for kv cache. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2017.

Qin, Y., Li, X., Zhang, J., and Chen, W. Cake: Cascading and adaptive kv cache eviction with temporal attention windows. *arXiv preprint arXiv:2503.01245*, 2025.

Sharma, A., Li, Y., and Liu, T. Minikv: Pushing the limits of 2-bit kv cache via compression and system co-design. In *Findings of the Association for Computational Linguistics: ACL*, 2025.

Shen, Y., Huang, S., Li, L., and Chen, W. Lava: Layer-wise kv cache eviction with dynamic budget allocation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025.

Sviridenko, M. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.

Tang, Z., Zhao, J., Zhang, H., Zhu, S., Wang, Y., and Zheng, Z. Squeezeattention: 2d management of kv-cache in llm inference via layer-wise optimal budget. In *International Conference on Learning Representations*, 2025.

Wan, Z., Wu, X., Yu, Y., Zhang, Z., Bi, X., et al. D2o: Dynamic discriminative operations for efficient generative inference of large language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Wang, X., Jiang, W., Li, L., Lyu, F., and Li, J. Dynamickv: Task-aware adaptive kv cache compression for long context llms. *arXiv preprint arXiv:2412.14838*, 2024.

Wang, Y., Zhang, L., Chen, Y., and Li, W. Kvtuner: Sensitivity-aware layer-wise mixed-precision kv cache quantization for efficient llm inference. In *International Conference on Machine Learning*, 2025.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2024.

Yang, H., Chen, S., Li, Z., and Xie, P. Xquant: Ultra-low bit kv cache quantization with cross-layer compression. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Yu, B. and Chai, Y. Evolkv: Evolutionary kv cache compression for efficient llm inference. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024.